Title: "10.2 Assignment: Recommender System (Movies)"
Class: "DSC630-T301 Predictive Analytics (2261-1)
Student: "Roshan GC"
Date: "11/16/2025"

Description:

In this assignment I will build a movie recommender system by using the 'MovieLens small dataset' given in assignment. User can type the name of the movie and in returns they will they will get list of 10 similar types movies recommendation. This system can be use by Netflix to personalize content and for user engagement. To build it I will use item based collaborative filtering with cosine similarity to suggest a movies to the user.

1 – Import Libraries

In [2]:
```
#importing the required libraries.
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
```

2 – Load the MovieLens Data set

I will load 1. movies.csv and 2. ratings.csv data to build a movie recommender.

In [3]:
```
# loading the datasets from local CSV files

movies = pd.read_csv("/Users/roshan/Downloads/DSC630/movieLens_Dataset/movie
ratings = pd.read_csv("/Users/roshan/Downloads/DSC630/movieLens_Dataset/rati

print("Movies data:")
display(movies.head())

print("Ratings data:")
display(ratings.head())
```

```
Movies data:
```

	movieId	title	genres
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
1	2	Jumanji (1995)	Adventure|Children|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	5	Father of the Bride Part II (1995)	Comedy

```
Ratings data:
```

	userId	movieId	rating	timestamp
0	1	17	4.0	944249077
1	1	25	1.0	944250228
2	1	29	2.0	943230976
3	1	30	5.0	944249077
4	1	32	5.0	943228858

3 – Merging Movies with Ratings Tables

I will merge the ratings and movies data by using the -movieId- common column they have. The reason behind it is movies table doesn't have ratings and ratings table doesn't have titles. This combination makes it easirer to build a movie recommender. I will create a new table after merge.

In [4]:
```
# Merging ratings and movies tables based on movieId
data = pd.merge(ratings, movies, on="movieId")

print("Merged data:")
display(data.head())
```

```
Merged data:
```

	userId	movieId	rating	timestamp	title	genres
0	1	17	4.0	944249077	Sense and Sensibility (1995)	Drama|Romance
1	1	25	1.0	944250228	Leaving Las Vegas (1995)	Drama|Romance
2	1	29	2.0	943230976	City of Lost Children, The (Cité des enfants p...	Adventure|Drama|Fantasy|Mystery|Sci-Fi
3	1	30	5.0	944249077	Shanghai Triad (Yao a yao yao dao waipo qiao) ...	Crime|Drama
4	1	32	5.0	943228858	Twelve Monkeys (a.k.a. 12 Monkeys) (1995)	Mystery|Sci-Fi|Thriller

4 – Building the User Movie Rating Matrix

I will merge the data into a user-move matrix where each row represents 'userId' and each column represents 'title'. Cell records the rating as per user input. I will replace the missing values wiht 0. Recommender system will rely on a matrix of users and movies. This matrix will show each movie as a vector of ratings and then measure the similarity between movives based on user rating.

In [15]:
```
# Since there is a lot of data so I am using only the first 50,000 rows
data_small = data.head(50000)

user_movie_matrix = data_small.pivot_table(
    index="userId",
    columns="title",
    values="rating"
).fillna(0)

print("Shape:", user_movie_matrix.shape)
user_movie_matrix.head()
```

```
Shape: (323, 8129)
```

Out[15]:

title	"Great Performances" Cats (1998)	#Alive (2020)	'burbs, The (1989)	(500) Days of Summer (2009)	*batteries not included (1987)	...And Justice for All (1979)	...First Do No Harm (1997)	1 (2014)
userId								
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
5	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

5 rows × 8129 columns

5 – Compute Movie–Movie Similarity (Cosine Similarity)

I will compute a similarity score between all pair of movies by using cosine similarity. In item based collaborative filtering the idea is movies could be similar if there is similarity in ratings by multiple users. we will consider it similar, if two movies tend to get similar patterns accross users. Cosine similarity gives us a natural way to measure how aligned two rating vectors are.

In [16]:
```
# Transposing the user–movie matrix so rows will become movies
movie_matrix = user_movie_matrix.T

# Computing cosine similarity
cosine_sim = cosine_similarity(movie_matrix, movie_matrix)

movie_similarity_df = pd.DataFrame(
    cosine_sim,
    index=movie_matrix.index,
    columns=movie_matrix.index
)

print("Movie/movie similarity matrix shape:", movie_similarity_df.shape)
display(movie_similarity_df.iloc[:5, :5])
```

```
Movie/movie similarity matrix shape: (8129, 8129)
```

title	"Great Performances" Cats (1998)	#Alive (2020)	'burbs, The (1989)	(500) Days of Summer (2009)	*batteries not included (1987)
title					
"Great Performances" Cats (1998)	1.000000	0.000000	0.000000	0.222988	0.000000
#Alive (2020)	0.000000	1.000000	0.333712	0.222988	0.000000
'burbs, The (1989)	0.000000	0.333712	1.000000	0.238597	0.568149
(500) Days of Summer (2009)	0.222988	0.222988	0.238597	1.000000	0.073819
***batteries not included (1987)**	0.000000	0.000000	0.568149	0.073819	1.000000

6 – Building the Recommender Function

I will create this recommender function and it will try find the exact movie title in our similarity matrix. This function will look up for similar scores for the chosen movie and it will return the top 10 most similar movies.

In [17]:
```
# creating function to recommend movies.

def recommend_movies(movie_name, n_recommendations=10):

    # Check if title exists
    if movie_name not in movie_similarity_df.index:
        print(f" Movie '{movie_name}' not is found in the dataset.")
        print("Here are some movie suggestions you can try:")
        print(movie_similarity_df.index[:20])
        return pd.DataFrame(columns=["recommended_title", "similarity"])

    # Getting similarity scores for the chosen movie
    similarity_scores = movie_similarity_df[movie_name].sort_values(ascendir

    # Removing the movie itself
    similarity_scores = similarity_scores.drop(labels=[movie_name])

    # Take top recommendation
    top_n = similarity_scores.head(n_recommendations)

    # Return result
    result = pd.DataFrame({
        "recommended_title": top_n.index,
        "similarity": top_n.values
    })
```

```
    return result
```

7 – Running the Recommender function

I will enter the movie name 'Inception (2010)' manually and run the fuction to find out if it returns the top 10 similar movies. This step will confirm our code is working as expected or not.

In [20]:
```
# Ask the user to type a movie_name
movie_name = "Inception (2010)"

print(f"Entered movie: {movie_name}\n")
recommendations = recommend_movies(movie_name, 10) # calling the recommend_m

print("Top 10 Recommended Movies List:")
display(recommendations)
```

```
Entered movie: Inception (2010)

Top 10 Recommended Movies List:
```

	recommended_title	similarity
0	Dark Knight, The (2008)	0.702925
1	Interstellar (2014)	0.698609
2	Avatar (2009)	0.653048
3	Shutter Island (2010)	0.646531
4	Prestige, The (2006)	0.634989
5	Matrix, The (1999)	0.623227
6	Fight Club (1999)	0.622572
7	Dark Knight Rises, The (2012)	0.618383
8	Lord of the Rings: The Return of the King, The...	0.616597
9	Wolf of Wall Street, The (2013)	0.613164

Note: Above result confirms Recommender System is working as expected and its recommending the movies based on similarity.

8 – Summary of the Recommender System (Movies)

This assignmet gives the basic type of Recommender system for the movies. It is collaborative and item-based cause uses patterns in user ratings and it focuses on

relationship between movies.

System like this is used in the real world application to suggest movies or any other products helping users to discover the new content so they can enjoy with our recommendations.